

Signed By Default Camp

2026-05-03

As with many discussions in the programming space, there are “wars” between different ways of doing things. These are typically about minor aesthetic preferences, such as:

- Tabs vs Spaces for indentation
- `snake_case` vs `camelCase` vs `Ada_Case` for naming conventions
- `'single quote'` vs `"double quote"` for strings (if the language allows both)
- 1TBS vs K&R vs Allman for `{ }` brace styles

These wars are largely pointless; what actually matters is coherency and consistency in your coding style. However, when it comes to designing a language, some binary choices have a massive impact. This article focuses on one such choice: whether to default to signed or unsigned integers.

A fellow language designer, Christoffer Lernö, of the [C3 language](#) has written an article regarding his decision to change from unsigned integers to signed integers as the default integer kind for C3: [Unsigned sizes: a five year mistake](#). I highly recommend the article as it does cover the discovery process and trade-offs that have to be made when designing a programming language.

The Different Camps

When designing **Odin**, I explicitly chose signed-by-default. I used to be in the unsigned-by-default camp many years before I even started Odin, but I've seen too many people make mistakes with unsigned arithmetic¹.

The most common problem is the mentality of using unsigned types to “enforce” that a value is never negative. The irony is that these same people do arithmetic assuming normal algebra rules apply, where subexpressions (e.g. the `a-b` part of `a-b+c`) can go negative even if the final result is positive. This leads to infinite loops and out-of-bounds errors. The “never negative” mentality is a fundamental misunderstanding of how integers work on the machine. You could call it a skill issue, but it's so widespread that I don't think that's a fair dismissal.

My colleague is in the **Almost Always Unsigned**² camp. My biggest disagreement with him on this topic is that staying in that camp requires being **highly** competent and careful with every single operation, and even most highly competent programmers aren't that careful all of the time. He knows all the edge cases and pathological cases, which I don't think most unsigned-by-default advocates actually do. But the ones who do are exactly the people for whom the default integer kind matters least.

I also defined all integer arithmetic in Odin to be wrapping, both unsigned and signed, along with defining the results of operations like `x / 0`³ and `INT_MAX - INT_MIN` so that they are not “undefined/illegal behaviour”.

¹ It could be argued that unsigned is better “in theory”, but signed works better “in practice”. Personally, if something doesn't work “in theory” in practice, then the “theory” was wrong to begin with and merely a false hypothesis.

² I highly recommend reading this article too to see the edge cases and pathological cases with signed integers.

³ The user can define division-by-zero behavior compiler-wide or file-wide

Lastly, it matters whether the language supports implicit numeric conversions. Many languages still perform implicit integer conversions, especially when no **value** information is lost. In Odin I disallowed even this, for many reasons, but a big one is that **type** information is lost even if the **value** is preserved. Making all conversions explicit makes intent clearer and gives the programmer the choice. As the article notes, many unsigned bugs in C arise from implicit conversions between signed and unsigned. Language design should account for the mistakes of past languages and understand the context they were made in, rather than reject entire ideas wholesale.

(trap, zero, self, or all-bits). The default is trapping, but there are good cases for defined division by zero, especially in proofing algorithms.

Possible Implicit Conversion Rules

Implicit integer conversion rules can be complex or very simplistic, and different rulesets lead to different outcomes. Take this example:

```
a: u8  = 53
b: u8  = 34
c: u16 = a * b
```

The question is: what happens to `a * b`? These are the possible approaches:

0. No implicit conversions (Odin-style)
1. Naïve unidirectional (bottom-up) type conversions
2. Convert to the “natural” integer size, then implicitly truncate (C-style)
3. Convert to the largest integer size, then implicitly truncate
4. Bidirectional (top-down) type conversions

Option 0 makes this a type error and forces the user to specify the intended behavior. Option 1⁴ either wraps to 8 bits (even though `53 * 34` fits in 16 bits) or treats the overflow as illegal or traps. Option 2 is similar to what C does: the operation becomes `u16(u32(a) * u32(b))`, preserving the most value information and likely performing better with a natural integer size. Option 3 is the same idea but uses a larger intermediate type such as `u64` or `u128`. Option 4 is my preferred approach if I wanted implicit conversions: propagate the type hint from the left-hand side top-down so that it serves as the widest type down the syntax tree, resulting in `u16(a) * u16(b)` directly.

⁴ Of these approaches, I believe option 1 to be the objectively worse approach to choose from when designing a language. It leads to the most bugs in practice if value information is wanted to preserved, and does not allow for any performance optimizations allowing for widening.

Overflow/Underflow, Wombling Free

Some languages, such as Rust, treat overflowing arithmetic as a trap in debug builds (with a significant runtime cost) but as wrapping in release builds. I don't like behavior that changes based on optimization level; I want things to behave the same unless I explicitly ask otherwise.

A separate question is which is worse when it happens: overflow or underflow, with signed or unsigned. I'd argue that overflow is far less common than going below zero for both signed and unsigned integers. And because of this, underflow on unsigned integers is the more common bug, and more likely to cause serious problems than simply having a negative value.

